Especificação Técnica: Aplicação FullStack de Gestão de Eventos e Ingressos (All Tickets)

1. Visão Geral e Stack

1.1 Objetivo

O **All Tickets** é uma aplicação FullStack para gerenciamento centralizado de eventos, comercialização e validação de ingressos digitais, além do controle de repasses financeiros compartilhados entre a plataforma e os produtores. O sistema oferece acesso segmentado para três perfis de usuários (`PUBLICO`, `EMPRESA` e `ADMIN`), garantindo a integridade transacional das vendas e prevenindo fraudes na entrada dos eventos via validação de *hashes* criptográficos.

1.2 Stack Tecnológica

- **Linguagem:** Python 3.11+ (Backend) e JavaScript ES6+ (Frontend)
- **Framework Web Backend:** Django com ORM e Django Templates com Bootstrap 5 (ou Single Page Application reativa)
- **Persistência / ORM:** PostgreSQL 16+
- **Segurança e Criptografia:** Criptografia nativa SHA-256 (`crypto.subtle` / `hashlib`) para armazenamento de senhas e geração de QR Codes/Hashes únicos de ingressos
- **Validação de Dados:** Pydantic v2

- **Testes Unitários e Integração:** Módulo `django.test` e `pytest`

- **Containerização:** Docker e Docker Compose

1.3 Arquitetura e Restrições

- **Padrão Arquitetural:** Padrão Django MVT (Model-View-Template) com camadas limpas de domínio:
    - `Domain`: Entidades puras (Usuário, Evento, Ingresso, Transação), Objetos de Valor e Exceções de Domínio.
    - `Use Cases`: Regras de negócio da aplicação (Checkout com split 80/20, Validação de Portaria, Aprovação de Parceria).
    - `Adapters / Controllers`: Controllers/Views do Django, DTOs de Entrada/Saída.
    - `Infrastructure`: Repositórios Django ORM, integração com PostgreSQL e serviços externos.
- **Controle de Acesso Baseado em Funções (RBAC):** Restrição estrita de navegação e execução de rotas de acordo com a função autenticada (`PUBLICO`, `EMPRESA`, `ADMIN`).

2. Entidades e Dados

2.1 Modelo de Dados (DDL SQL)

```
SQL
-- Tabela de Usuários e Autenticação (TB_USUARIOS)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(64) NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'PUBLICO', -- 'PUBLICO',
'EMPRESA', 'ADMIN'
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Tabela de Solicitações de Parceria / Vinculação Empresarial
(TB_SOLICITACOES_VINCULACAO)
CREATE TABLE company_binding_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_name VARCHAR(120) NOT NULL,
    cnpj VARCHAR(18) NOT NULL,
    email VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'PENDENTE', -- 'PENDENTE',
'APROVADO', 'REJEITADO'
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Tabela de Eventos e Publicações (TB_EVENTOS)
CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(120) NOT NULL,
    category VARCHAR(50) NOT NULL, -- 'Shows e Festivais', 'Teatro e
Cultura', 'Festas e Nightlife'
    price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
    stock INT NOT NULL CHECK (stock >= 0),
    likes_count INT DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Tabela de Ingressos Comprados (TB_INGRESSOS)
CREATE TABLE tickets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    qr_hash VARCHAR(64) UNIQUE NOT NULL,
    is_used BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    used_at TIMESTAMP WITH TIME ZONE
);

-- Tabela de Livro Caixa e Transações Financeiras (TB_FINANCEIRO)
CREATE TABLE financial_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ticket_id UUID UNIQUE NOT NULL REFERENCES tickets(id) ON DELETE
CASCADE,
    total_amount DECIMAL(10, 2) NOT NULL,
    platform_share_20 DECIMAL(10, 2) NOT NULL,
    company_share_80 DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

2.2 Relacionamentos

- **User -> CompanyBindingRequest:** 1 Usuário pode enviar N Solicitações de Vinculação para transformar sua conta em Empresa (1:N).
- **User (Empresa) -> Event:** 1 Produtor possui N Eventos cadastrados (1:N).
- **Event -> Ticket:** 1 Evento possui N Ingressos vendidos (1:N). Ao excluir um evento, seus ingressos associados são tratados via cascata.
- **User (Comprador) -> Ticket:** 1 Usuário Comprador possui N Ingressos em sua carteira digital (1:N).

- **Ticket -> FinancialTransaction:** Cada Ingresso gerado gera 1 Transação Financeira de divisão de valores (1:1).

3. Interface Gráfica WEB (Django Templates + Bootstrap 5 / SPA)

A aplicação disponibiliza uma interface visual unificada renderizada e dinamizada para três perfis de acesso:

3.1 Painéis e Visões

- **Feed de Eventos Público (`/` ou `/public`):**
    - Catalogo visual de cards de eventos disponíveis com imagem, badge de categoria, organizador, preço e estoque em tempo real.
    - Ações de interação ("Curtir") e botão de disparo do Checkout de compra de ingressos.
    - Acesso ao modal de **"Solicitar Vinculação de Empresa"** para submissão de CNPJ e proposta.
    - Acesso à **"Carteira Digital"** exibindo os bilhetes comprados pelo usuário autenticado com seu respectivo QR Hash criptográfico.
- **Painel do Produtor (`/company`):**
    - **Barra de Métricas (KPIs):** Total de ingressos vendidos, receita bruta acumulada e saldo líquido a receber (80%).
    - **Formulário de Cadastro de Eventos:** Adição de novos títulos, categorias, preços e estoque.

    - **Módulo Validador de Portaria:** Campo de checagem imediata de código Hash/QR Code para liberação de entrada e queima do ingresso.

- **Painel Administrativo (`/admin`):**
    - **Barra de Métricas Globais:** Volume total transacionado na plataforma e comissão acumulada da All Tickets (20%).
    - **Tabela de Gestão de Usuários:** Interface para promoção direta de papéis em tempo real (`PUBLICO` ➔ `EMPRESA` / `ADMIN`).
    - **Fila de Solicitações de Vinculação:** Tabela de auditoria para aprovar ou rejeitar propostas de novas empresas.

4. Contratos de API REST (v1)

4.1 Cadastrar Novo Evento

- **Endpoint:** `POST /api/v1/events`

- **Descrição:** Permite que uma empresa parceira cadastre uma nova atração.
- **Headers:** `Content-Type: application/json`, `Authorization: Bearer <token_empresa>`
- **Request Body:**

JSON

```
{
  "title": "Maceió Summer Sounds 2026",
```

```
  "category": "Shows e Festivais",
  "price": 100.00,
  "stock": 50
}
```

- **Respostas:**
    - `201 Created`:

JSON

```
{
  "id": "e3a8910d-2b4a-4f51-b883-92f701b918a2",
  "company_id": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "title": "Maceió Summer Sounds 2026",
  "category": "Shows e Festivais",
  "price": 100.00,
  "stock": 50,
  "likes_count": 0,
  "created_at": "2026-09-17T09:00:00Z"
}
```

- `403 Forbidden`: Usuário não possui o papel `EMPRESA`.
- `422 Unprocessable Entity`: Dados inválidos ou preço/estoque negativos.

4.2 Comprar Ingresso (Checkout & Split Financeiro)

- **Endpoint:** `POST /api/v1/events/{event_id}/purchase`
- **Descrição:** Realiza a compra do ingresso, reduz o estoque, gera o QR Hash e executa a divisão 80/20.
- **Path Parameter:** `event_id` (UUID)
- **Request Body:**

JSON

```
{
  "payment_method": "PIX"
}
```

- **Respostas:**
    - `201 Created`:

JSON

```
{
  "ticket": {
    "id": "8f3b20a1-9d11-4c62-81e3-2e008a129999",
    "event_title": "Maceió Summer Sounds 2026",
    "qr_hash": "QR-A9F82B10C4",
    "is_used": false,
    "created_at": "2026-09-17T09:10:00Z"
  },
  "financial_split": {
    "total_amount": 100.00,
    "company_share_80": 80.00,
    "platform_share_20": 20.00
  }
}
```

- `400 Bad Request`: Evento com estoque esgotado.
- `401 Unauthorized`: Usuário não autenticado.

4.3 Validar Ingresso na Portaria

- **Endpoint:** `POST /api/v1/tickets/validate`
- **Descrição:** Valida a autenticidade do bilhete e altera seu estado para utilizado.
- **Request Body:**

JSON

```
{
  "qr_hash": "QR-A9F82B10C4"
}
```

- **Respostas:**
    - `200 OK`:

JSON

```
{
  "status": "VALIDATED",
  "message": "Entrada liberada com sucesso!",
  "event_title": "Maceió Summer Sounds 2026",
  "validated_at": "2026-09-17T20:30:00Z"
}
```

- `400 Bad Request`: Ingresso já utilizado anteriormente (`USED_TICKET`).
- `404 Not Found`: Código Hash inexistente ou inválido.

5. Regras de Negócio e Casos de Borda

5.1 Regras de Negócio (RN)

- **RN-01 (Divisão Financeira Automática):** Toda compra efetuada deve ter seu valor total automaticamente dividido em **80% de saldo para a empresa produtora** e **20% de comissão retida para a plataforma All Tickets**.

- **RN-02 (Controle Rigoroso de Estoque):** A compra só pode ser autorizada se o evento possuir estoque maior que zero (`stock > 0`). A confirmação da compra deve decrementar 1 unidade do estoque atomicamente.

- **RN-03 (Unicidade e Criptografia do QR Hash):** Todo ingresso emitido deve possuir uma hash SHA-256 única e inviolável para apresentação na portaria.

- **RN-04 (Prevenção de Reuso de Ingressos / Antifraude):** Um ingresso só pode ser validado uma única vez na entrada do evento. Após a leitura, a flag `is_used` é alterada para TRUE e tentativas subsequentes de leitura devem ser rejeitadas.

- **RN-05 (Promoção e Alteração de Permissões):** Somente usuários com a função `ADMIN` têm permissão para aprovar solicitações de vinculação empresarial e alterar os papéis (`role`) de outros usuários no sistema.

5.2 Casos de Borda (CB)

- **CB-01 (Estoque Esgotado no Momento da Compra):** Tentar comprar um ingresso com `stock = 0` deve ser interrompido e retornar `400 Bad Request` com mensagem `"Ingressos esgotados para este evento"`.

- **CB-02 (Tentativa de Fraudística de Reutilização de Bilhete):** Enviar um `qr_hash` de um ingresso cuja flag `is_used` já seja TRUE deve retornar `400 Bad Request` com erro de domínio `"ALERTA: Ingresso já utilizado anteriormente na portaria"`.

- **CB-03 (Acesso Não Autorizado a Rotas Restritas):** Um usuário com papel `PUBLICO` tentando acessar endpoints do painel `/company` ou `/admin` deve ser bloqueado com `403 Forbidden`.

- **CB-04 (Hash Inexistente):** Tentar validar um `qr_hash` não cadastrado no banco deve retornar `404 Not Found`.

6. Critérios de Aceite (Exemplos para Testes Automatizados)

Estes cenários devem orientar a automação de testes com `pytest` e `httpx`.

Cenário 1: Sucesso na Compra de Ingresso com Divisão Financeira 80/20

- **Dado** que existe um evento publicado com `id = "e3a8910d-2b4a-4f51-b883-92f701b918a2"`, `price = 100.00` e `stock = 10`
- **Quando** um usuário autenticado envia uma requisição POST para `/api/v1/events/e3a8910d-2b4a-4f51-b883-92f701b918a2/purchase`
- **Então** o código de status HTTP retornado deve ser `201 Created`
- **E** o estoque restante do evento deve ser atualizado para 9
- **E** deve ser gravada uma transação financeira com `company_share_80 = 80.00` e `platform_share_20 = 20.00`
- **E** o ingresso gerado deve estar disponível na carteira do usuário com status `is_used = False`.

Cenário 2: Validação de Portaria e Bloqueio de Reuso (Antifraude)

- **Dado** um ingresso salvo no banco com `qr_hash = "QR-A9F82B10C4"` e `is_used = False`

- **Quando** a empresa envia uma requisição POST para `/api/v1/tickets/validate` com o hash `"QR-A9F82B10C4"`

- **Então** a resposta deve ser `200 OK` e o status do bilhete deve mudar para `is_used = True`

- **E quando** for realizada uma **segunda** requisição POST utilizando o mesmo `qr_hash = "QR-A9F82B10C4"`

- **Então** o status HTTP deve ser `400 Bad Request`

- **E** o corpo da resposta deve conter a mensagem de erro `"ALERTA: Ingresso já utilizado anteriormente na portaria"`.

Cenário 3: Aprovação de Solicitação e Promoção de Usuário para Empresa

- **Dado** uma solicitação de parceria registrada com `id = "REQ-101"` e `status = "PENDENTE"` pertencente a um usuário com papel `"PUBLICO"`

- **Quando** o administrador autenticado envia uma requisição PATCH para `/api/v1/company-requests/REQ-101` com `{"status": "APROVADO"}`

- **Então** o código de status deve ser `200 OK`

- **E** o papel (`role`) do usuário solicitante deve ser promovido para `"EMPRESA"` na tabela `users`.